



INTERNATIONAL UNIVERSITY OF GRAND-BASSAM  
Excellence • Accountability • Opportunity

SCHOOL OF SCIENCE AND ENGINEERING

CSC 4305 · Software Engineering · Fall 2026

# What Software Engineering Is

*Nine real programmes, eight of them failures, and not one caused by bad code*

Session 1 of 28 · Tuesday 8 September 2026 · 1:00 – 2:20 PM

· Prof. Guillaume Sanje Mpacko, Ph.D.

SESSION 1 OF 28

## What Software Engineering Is

Tuesday, 8 September 2026 · 1:00 – 2:20 PM · Unit 1 · Foundations and Process

### LEARNING OBJECTIVES

- ✓ Distinguish programming from software engineering on three concrete axes — time, scale and consequence.
- ✓ Explain the origin of the term *software crisis* and why it is still live, citing two documented failures.
- ✓ Name the five classical life-cycle phases and the principal artifact each produces.
- ✓ Classify a system as generic, bespoke, embedded or data-intensive, and say why the class changes who owns the requirements.
- ✓ State the four kinds of maintenance, and why maintenance — not construction — dominates lifetime cost.
- ✓ Diagnose a documented failure by separating its proximate technical cause from the process failure that let the cause reach production.

**Everyone in this room can already program.** Nobody in this room has yet built a system that outlives the person who wrote it. Today we look at nine real programmes — consumer products, aircraft, an airport, an airline, and a 500-line script — and extract the discipline that separates the ones that survived from the ones that cost billions. **Not one of the failures was caused by bad code.**

## 1 • What “engineering” actually means

---

Before defining software engineering, define engineering — using a discipline that has been accountable for its decisions far longer than ours has. Everyone in this room can already treat a headache. Given a pharmacy, a search engine and enough confidence, you could hand someone a tablet that makes the pain go away. Nobody would call that medicine, and the reason has nothing to do with whether the tablet worked.

### **Making something**

A gifted healer may cure one patient better than a hospital can. Skill is real, and it is not the thing in question.

### **Engineering something**

What the healer cannot do is guarantee the ten-thousandth patient, or explain a decision to a coroner four years later.

### It predicts failure

BEFORE IT HAPPENS

A clinician does not wait for the heart attack to find out whether the patient was at risk. She measures, scores, treats, and follows up to confirm what she already believed.

A test that fails is engineering working. An outage caused by a case nobody enumerated is engineering that did not happen.

### It makes decisions traceable

TO REASONS

Every line on a prescription traces to a finding, a guideline, or a decision someone signed.

This is not bureaucracy. It is what lets you change one thing five years later without silently breaking three others — and answer a court that asks who knew what.

### It accepts the constraints

AS NON-NEGOTIABLE

Dosage limits, interactions, consent law, what the pharmacy stocks, the hours the nurse can work, what the patient can afford. None were chosen by the clinician.

The medicine consists precisely of finding a treatment that satisfies all of them at once.

### THE WHOLE COURSE IN ONE SENTENCE

Programming is producing behaviour that works. Engineering is producing behaviour that **keeps working**, under constraints you did not choose, for people who will not forgive you, in a way that someone else can change after you have gone.

## 2 • Nine programmes: one that worked, eight that did not

Every figure below can be checked against the sources at the end. Where reported totals differ, the more conservative number is used. Read the last column first.

| Programme | Outcome | What actually failed | The lesson this course draws |
|-----------|---------|----------------------|------------------------------|
|-----------|---------|----------------------|------------------------------|

|                                        |                          |                                                                                                                                                                                    |                                                                                                                                              |
|----------------------------------------|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Dacia Logan</b><br>1999–present     | <b>1.8 m sold</b>        | Nothing. A €5,000 price was fixed <i>first</i> and everything else negotiated against it.                                                                                          | A constraint stated first and treated as non-negotiable produces better engineering than an open-ended wish list.                            |
| <b>Ford Edsel</b><br>1957–1959         | <b>\$250 m lost</b>      | Built exactly as specified. The research was for the market of 1955; the car arrived in 1957, into a recession.                                                                    | Requirements have a shelf life. A specification agreed eighteen months ago and never revisited is a historical document.                     |
| <b>Tata Nano</b><br>2009–2018          | <b>518 sold in 2018</b>  | Hit an extraordinary engineering target, then was sold as “the cheapest car in the world” in a market where a car is a public statement.                                           | Elicitation is not transcription. What a stakeholder asks for and what would satisfy them diverge constantly.                                |
| <b>GM ignition switch</b><br>2002–2015 | <b>124 deaths</b>        | A \$0.57 part approved although the supplier said it missed GM’s own torque specification. Change made quietly, records not updated, safety failure logged as a convenience issue. | A requirement waived without a record is a defect with a delayed fuse. This is what traceability is for.                                     |
| <b>VW defeat device</b><br>2008–2015   | <b>€33 bn+</b>           | Software that detected test conditions and ran clean. It passed every test it was given.                                                                                           | Tests are evidence about the goal, not the goal. Optimising for the measurement is an ethics failure before it is a technical one.           |
| <b>A380 wiring</b><br>2005–2007        | <b>~2 yr delay</b>       | Two design teams on two versions of one design tool. Harness geometry did not survive the transfer; 530 km of cable per aircraft did not fit.                                      | Configuration management at industrial scale — the same failure class as two developers building against different versions of an interface. |
| <b>A380 vs 787</b><br>2000–2021        | <b>251 of ~700 built</b> | A magnificent machine built for a hub-to-hub future that airlines did not adopt. Boeing bet point-to-point and was right.                                                          | Architecture encodes a prediction about the world. When the prediction is wrong, excellence inside the architecture rescues nothing.         |
|                                        |                          |                                                                                                                                                                                    |                                                                                                                                              |

**737 MAX /  
MCAS**  
2018–2020

**346  
deaths**

A rational commercial requirement — no new pilot type-training — met by software reading **one** angle-of-attack sensor, with authority to trim nose-down, omitted from the manuals.

Two lessons: a single point of failure in a safety-critical input is a design defect however reliable the input usually is; and a system hidden from its operators has left engineering for ethics.

**Denver baggage**  
1991–1995

**\$560 m  
over**

Scope grew from one concourse and one airline to three and twenty, without re-baselining cost or schedule. Testing compressed by pressure to open. No fallback.

Three failures you will be tempted to repeat: scope changed without re-planning, testing as the schedule's shock absorber, and no way back.

**Southwest 2022**  
software from  
2004

**16,700  
flights**

A crew-scheduling system built for an operation two-thirds the size, handling ~300 schedule changes per batch. Internal warnings raised years earlier; modernisation deferred.

Technical debt is a real liability with a real interest rate, and it is paid in one lump sum on the worst day of the year.

VALIDATION

**Right thing, built badly**

Denver baggage · Southwest 2022

*Expensive. Recoverable.*

**Right thing, built well**

Dacia Logan · Boeing 787

*The only quadrant that pays.*

**Wrong thing, built badly**

Mercifully rare —  
these get cancelled early

**Wrong thing, built well**

Edsel · Nano · A380 · MCAS · VW

**No amount of testing saves these.**

VERIFICATION — built it right? no ← → yes

*The lower-right quadrant holds the most expensive failures in this document.*

**Figure 1.** Two questions, four outcomes. Verification asks whether the product conforms to its specification. Validation asks whether the specification was the right one. A system can pass verification perfectly and still be a total failure — and those are the cases nobody knows how to test their way out of.

## 2.1 What the eight failures have in common

1

**A requirement nobody validated.** The Edsel and the Nano were built exactly as specified, for buyers who wanted something else.

- 2 **An architectural bet nobody revisited.** The A380 was a superb answer to a question airlines answered differently.
- 3 **A specification waived without a record.** The GM switch went into production known not to meet its own standard.
- 4 **A single point of failure in a critical path.** MCAS trusted one sensor, and 346 people died.
- 5 **Scope that grew while the plan stood still.** Denver added two concourses and twenty airlines without moving a date.
- 6 **A known weakness deferred until the worst possible day.** Southwest knew about its scheduler for eighteen years.

#### THE ANSWER TO THE QUESTION IN THE TITLE

Every one of those six is a topic on this syllabus, and **none of them is a coding skill**. The engineering is not in the construction. It is in the deciding, the recording, the predicting and the accounting — all of which happen before and around the construction, and none of which the construction can compensate for.

Every failure above was cheap to prevent and ruinous to correct. The gap between those two numbers is where this discipline lives.

#### Why this matters more here, not less

A team with money can absorb a bad decision by throwing another year at it. A health post serving 6,000 people on intermittent power cannot. Constraint is not the enemy of engineering — it is the reason engineering exists, and the Logan is the proof that designing to a hard constraint beats wishing it away.

## 3 · The discipline, and its vocabulary

### 3.1 The problem the discipline was invented to solve

In October 1968 a NATO working group met at Garmisch and deliberately chose a provocative title for the conference: *Software Engineering*. The word *engineering* was aspirational, not descriptive. Hardware capability was doubling every few years while the ability to produce reliable software for it was not

improving at all. Projects were late, over budget and defective on delivery. The phrase **software crisis** dates from that meeting.

Nearly sixty years later the crisis has changed shape rather than disappeared. Southwest's scheduler is a 2022 instance of a 1968 diagnosis. Everything in this course is the accumulated professional response to that pattern.

### 3.2 Programming is not software engineering

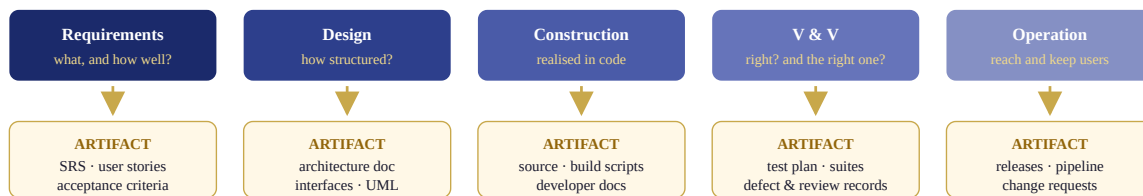
| Axis        | Programming                                          | Software engineering                                                                                                           |
|-------------|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Time        | The code must work now. Correctness is judged today. | It must keep working for years, under changing requirements, on changing platforms, maintained by people who did not write it. |
| Scale       | One author, one head, one file, thousands of lines.  | Many authors, many modules, hundreds of thousands of lines. No single person holds the whole system in mind.                   |
| Consequence | A defect costs the author an afternoon.              | A defect costs money, reputation, safety or lives. It must be traced, reported, prioritised, fixed, tested and released.       |

#### ONE-SENTENCE FORMULATION

Adapted from Titus Winters at Google: **software engineering is programming integrated over time**. Requirements, design, testing, configuration management, process — all of it exists because of that integration.

### 3.3 The classical life cycle and its artifacts

Whatever process model a team adopts, the same five kinds of work must be done. The models differ in how the work is ordered, batched and repeated — not in whether it happens. Each phase produces a durable artifact, and **the artifact is what survives when the people move on**.



*The artifact is what survives when the people move on.*

**Figure 2.** Five phases, five artifacts. Sessions 3 and 4 show five different ways of ordering exactly this work; the artifacts stay the same.

## TERMINOLOGY YOU WILL BE EXAMINED ON

**Verification** asks whether the product conforms to its specification.

**Validation** asks whether the specification was the right one. A system can pass verification perfectly and still be a failure — the A380, the Edsel and the Nano all did. That is the single most expensive mistake in the discipline, and it is the lower-right quadrant of Figure 1.

## 3.4 Categories of software, and why the category changes everything

### Generic products

Built speculatively for a market the developer defines — Excel, Android, PostgreSQL. **The developer owns the specification.** Change is driven by competition.

### Bespoke systems

Commissioned by one customer — a payroll system, a patient registry. **The customer owns the specification.** Change is driven by negotiation.

### Embedded systems

Control physical devices — brakes, pacemakers, irrigation controllers. Timing and reliability dominate; deployment is expensive or impossible to repeat.

### Data-intensive and AI-enabled

Behaviour derives from data, not from code alone. Their specification problem is unusually hard, because the intended behaviour cannot be fully written down in advance.

### Ask the category question first

It determines who has authority over the requirements, how tolerant the system is of defects, and how fast you can ship a correction. Which category is SANTE-BASSAM? Which was MCAS? Your answers should differ, and the difference should change how you would work on each.

## 4 · Where the money actually goes

Students consistently underestimate maintenance. Between **60% and 80%** of the lifetime cost of a long-lived system is spent after first delivery — and it divides into four buckets, only one of which is the one students expect.

| Bucket            | What it is                                                                                           | The part that surprises people                                                                               |
|-------------------|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <b>Corrective</b> | Fixing defects.                                                                                      | Typically the <i>smallest</i> share, around 20% of maintenance — and the only one most students think about. |
| <b>Adaptive</b>   | Responding to a change in the environment: a new OS version, a new tax rate, a new mobile-money API. | Nothing about your system changed. The world moved.                                                          |
| <b>Perfective</b> | Adding or refining functionality because users asked.                                                | Usually the largest share by a wide margin.                                                                  |
| <b>Preventive</b> | Restructuring code to keep future change affordable.                                                 | This is paying down technical debt. Southwest deferred it for eighteen years.                                |

### WORKED EXAMPLE

*What a 500-line script really costs*

A student writes a 500-line Python script that generates end-of-semester grade reports for one department. It works. The department adopts it. Five years later, what did it cost? We count engineer-days, and only work that would not have existed if the script had been discarded after one use.

| Bucket                  | What happened                                                                             | Days |
|-------------------------|-------------------------------------------------------------------------------------------|------|
| Construction            | Initial writing and debugging                                                             | 4    |
| Corrective              | Crash on a student with no middle name; division by zero for a course with no graded work | 3    |
| Adaptive                | Python 3.9 → 3.12; the registrar changes the CSV export format twice                      | 6    |
| Perfective              | GPA computation, then a second grading scale, then PDF output                             | 14   |
| Knowledge transfer      | Original author graduates; two successors each reconstruct intent from unreadable code    | 8    |
| Five-year lifetime cost |                                                                                           | 35   |



The eight days of knowledge transfer were the price of no requirements document, no tests, no README and no module structure. Roughly **two days** of discipline at the start — a short specification, a test file, named functions with docstrings — would plausibly have removed six of those eight, and reduced the corrective bucket as well.

### SESSION 1 TAKEAWAY

Not one of nine documented disasters was caused by a programmer who could not program. They were caused by a requirement nobody validated, an architectural bet nobody revisited, a specification waived without a record, a single point of failure, scope that grew while the plan stood still, and a known weakness deferred. Engineering discipline is not overhead imposed on programming — it is an investment whose return is measured over the life of the system, which is exactly why its value is invisible at the moment you must decide to pay for it.

## 5 · In-class exercise · failure post-mortem

---

In pairs, fifteen minutes, then sixty seconds each to report. You may search. Each pair takes one case.

### The cases

#### Pair A

Ariane 5 Flight 501 (1996) — the launcher lost 37 seconds after lift-off

#### Pair B

Therac-25 (1985–87) — a radiation machine that gave massive overdoses

#### Pair C

UK Post Office Horizon (1999–2019) — hundreds of sub-postmasters wrongly prosecuted

1. **The proximate technical cause**, in one sentence. What did the code do wrong?
2. **The process failure** that let that cause reach production. Name the life-cycle phase from §3.3 where it should have been caught.
3. **One practice from this syllabus** that would most plausibly have prevented it.

#### DO NOT READ THIS UNTIL YOU HAVE WRITTEN ALL THREE

In every one of these cases the proximate technical cause is small and comprehensible — an unhandled conversion, a race condition, a reused flag, a unit mismatch, a single sensor. The interesting failure is always organisational: a review that did not happen, a test environment that did not match production, a reused component whose assumptions were never re-validated in the new context, a warning logged in the wrong category.

**A pair that reports only the code bug has not finished the exercise.** That is the same distinction the whole first half of today was making: the code was fine.

---

## 6 · Six questions before your first line of code

---

Take these with you. Your team should be able to answer them in writing — not perfectly, but in writing, with names attached, so that in November you can tell whether you were right.

### Q1 — Who is this for?

AND WHAT WILL THEY STOP DOING?

Name a person, not a category. What do they do today, and which habit will the system replace?

### Q2 — What must be true

FOR THIS TO BE WORTH BUILDING?

State the number — patients registered, minutes saved, errors avoided — that would let someone judge you in six months.

### Q3 — Which constraints

ARE NOT NEGOTIABLE?

Power, connectivity, language, literacy, the device in the clerk's hand, the law on health records.

### Q4 — Which decision

IS MOST EXPENSIVE TO REVERSE?

That is your architecture. Write it down with its reason, and date it.

### Q5 — What is the single

POINT OF FAILURE?

If one component, one sensor, one person, one file is unavailable — what happens?

### Q6 — What is the fallback

WHEN THE CLEVER THING FAILS?

Denver had none. Your registry will run in a building where the power fails weekly.

None of these can be answered by writing code, and all of them get harder and more expensive to answer the longer you wait. That is the whole argument for designing before implementing, and every case in this lecture is evidence for it. **Friday we answer all six for SANTE-BASSAM.**

---

## 7 · Course setup · before Friday

### There is no homework for Sessions 1 and 2.

Individual assignments begin with Session 3. What follows is setup — do it once, and it takes about thirty minutes.

#### THREE THINGS, BEFORE SESSION 2 ON FRIDAY 11 SEPTEMBER

##### GitHub

Create an account if you do not have one and accept the two invitations: the `CSC_4305_SWE-Assignments` repository, which is where every template comes from, and your own private `CSC_4305_SWE-Student-FirstName-LastName` repository, which is where your work goes. Confirm you can see both.

##### Reading

Sommerville, *Software Engineering* 10th ed., Chapter 1 (or Pressman, Chapter 1). Come on Friday able to name the four attributes of good software that Sommerville lists.

##### One thought

Bring to Friday, in your head or on paper: **one software system you use daily that you consider well engineered**, and one sentence on why. We will use these in the first five minutes.

**The team.** There are six of you, so there is one team, and it is all of you. Friday sets up how it works — roles, repository, rhythm, and the fortnightly visit to the health post.

**The semester project — SANTE-BASSAM.** From Friday onward you build a patient registry and appointment system for a community health post serving roughly 6,000 people, on intermittent electricity and unreliable connectivity. You will take it through requirements, design, implementation, testing and delivery. The six questions above are your first deliverable against it.

## Key terms

software crisis

software life cycle

artifact

verification

validation

design-to-cost

generic product

bespoke system

embedded system

data-intensive system

corrective maintenance

adaptive maintenance

perfective maintenance

preventive maintenance

technical debt

single point of failure

traceability

## Readings and references

---

- Sommerville, I. *Software Engineering*, 10th ed., Pearson, 2016 — Chapter 1.
- Pressman, R. & Maxim, B. *Software Engineering: A Practitioner's Approach*, 9th ed., McGraw-Hill, 2020 — Chapter 1.
- Naur, P. & Randell, B. (eds). *Software Engineering: Report on a Conference Sponsored by the NATO Science Committee*, Garmisch, 1968 — read the discussion sections, not the papers.
- Leveson, N. & Turner, C. “An Investigation of the Therac-25 Accidents”, *IEEE Computer* 26(7), 1993.
- Winters, T., Manshreck, T. & Wright, H. *Software Engineering at Google*, O'Reilly, 2020 — Chapter 1.

**Sources for the case material.** Renault Group (Logan design-to-cost); published accounts of the Edsel programme and of Nano production and sales; the Valukas report and GM recall record (ignition switch); court and settlement records (Volkswagen); industry reporting on the A380 CATIA version divergence; CAPA analysis (A380 against 787); US Department of Justice statement and the two accident reports (737 MAX); Callear's reconstruction and the SEBoK entry (Denver); US DOT consent order and industry reporting (Southwest 2022). Where totals differ between sources, the more conservative figure has been used.